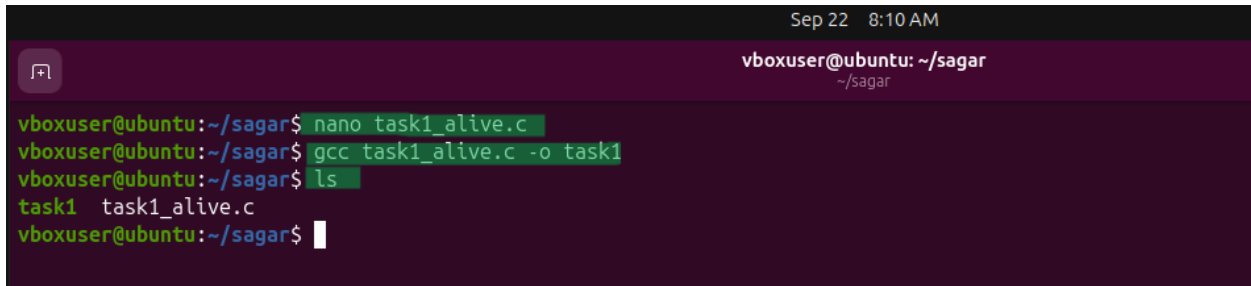


Table of Content

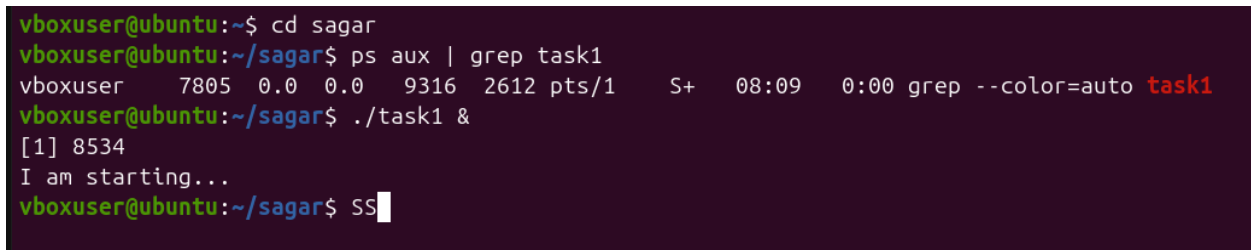
THE LONG-RUNNING PROCESS	2
Figure 1: Long running process part1	2
Figure 2: Long running process part2	2
PROCESS IDENTITY (PID AND PPID)	3
Figure 3: PROCESS IDENTITY (PID AND PPID)	3
EXIT CODES AND OS FEEDBACK	3
Figure 4: EXIT CODES AND OS FEEDBACK	4
STANDARD I/O STREAMS	4
Figure 5: STANDARD I/O STREAMS	4
CONDITIONAL EXECUTION AND TERMINATION	5
Figure 5: CONDITIONAL EXECUTION AND TERMINATION	5

THE LONG-RUNNING PROCESS



```
Sep 22 8:10 AM
vboxuser@ubuntu: ~/sagar
vboxuser@ubuntu:~/sagar$ nano task1_alive.c
vboxuser@ubuntu:~/sagar$ gcc task1_alive.c -o task1
vboxuser@ubuntu:~/sagar$ ls
task1 task1_alive.c
vboxuser@ubuntu:~/sagar$
```

Figure 1: Long running process part1



```
vboxuser@ubuntu:~$ cd sagar
vboxuser@ubuntu:~/sagar$ ps aux | grep task1
vboxuser  7805  0.0  0.0  9316  2612 pts/1    S+   08:09   0:00 grep --color=auto task1
vboxuser@ubuntu:~/sagar$ ./task1 &
[1] 8534
I am starting...
vboxuser@ubuntu:~/sagar$ ss
```

Figure 2: Long running process part2

Summary

I created a long-running C process that runs for 30 seconds and monitored it using the `ps` command.

PROCESS IDENTITY (PID AND PPID)

```
vboxuser@ubuntu:~/sagar$ nano task2_identity.c
vboxuser@ubuntu:~/sagar$ gcc task2_identity.c -o task2
vboxuser@ubuntu:~/sagar$ ./task2 &
[1] 7978
My PID is: 7978
My Parent PID is: 6562
Sleeping for 20 seconds...
vboxuser@ubuntu:~/sagar$ ps -p 7978 -o pid,ppid,cmd
  PID   PPID  CMD
[1]+  Done                  ./task2
vboxuser@ubuntu:~/sagar$
```

Figure 3: PROCESS IDENTITY (PID AND PPID)

Summary

I used `getpid()` and `getppid()` to find the process ID (PID) and parent process ID (PPID), and verified them using the `ps` command

EXIT CODES AND OS FEEDBACK

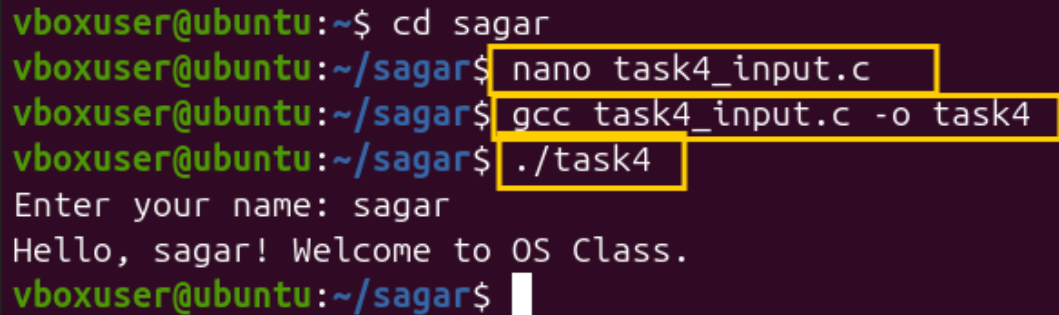
```
vboxuser@ubuntu:~/sagar$ nano task3_exit.c
vboxuser@ubuntu:~/sagar$ gcc task3_exit.c -o task3
vboxuser@ubuntu:~/sagar$ ./task3
Enter a number (positive for success, negative for fail): 5
Success
vboxuser@ubuntu:~/sagar$ echo $?
0
vboxuser@ubuntu:~/sagar$ ./task3
Enter a number (positive for success, negative for fail): -5
Failure
vboxuser@ubuntu:~/sagar$ echo $?
1
vboxuser@ubuntu:~/sagar$
```

Figure 4: EXIT CODES AND OS FEEDBACK

Summary

I created a program that returns different exit codes depending on the input, where 0 represents success and 1 represents failure, and checked the result using `echo $?`.

STANDARD I/O STREAMS



```
vboxuser@ubuntu:~$ cd sagar
vboxuser@ubuntu:~/sagar$ nano task4_input.c
vboxuser@ubuntu:~/sagar$ gcc task4_input.c -o task4
vboxuser@ubuntu:~/sagar$ ./task4
Enter your name: sagar
Hello, sagar! Welcome to OS Class.
vboxuser@ubuntu:~/sagar$
```

Figure 5: STANDARD I/O STREAMS

Summary

I demonstrated standard input and output by taking the user's name using `scanf()` and displaying it using `printf()`

CONDITIONAL EXECUTION AND TERMINATION

```
vboxuser@ubuntu:~/sagar$ nano task5_control.c
vboxuser@ubuntu:~/sagar$ gcc task5_control.c -o task5
vboxuser@ubuntu:~/sagar$ ./task5
Current PID: 8374
Do you want to continue? (1 for Yes, 0 for No): 1
Continuing...
vboxuser@ubuntu:~/sagar$ echo $?
0
vboxuser@ubuntu:~/sagar$ ./task5
Current PID: 8388
Do you want to continue? (1 for Yes, 0 for No): 0
Exiting...
vboxuser@ubuntu:~/sagar$ echo $?
1
vboxuser@ubuntu:~/sagar$
```

Figure 5: CONDITIONAL EXECUTION AND TERMINATION

Summary

I created a program that displays its PID, asks the user whether to continue, and uses conditional execution, `sleep()`, and exit codes to control the process.